Universidad Carlos III de Madrid

Escuela Politécnica Superior

Sistemas Distribuidos – Curso 2025-2026

Ejercicio Evaluable 1: Colas de Mensajes

Pablo Roig Sanchez, 100525980

Marcos Rodriguez Cuadrado, 100522136

Grupo: 80

Marzo 2026

1. Estructura de datos (claves.c)

Para almacenar las tuplas <key, value1, value2, value3> se ha utilizado una lista enlazada simple. Cada nodo de la lista es un struct claves con los siguientes campos:

- key[256] – cadena de caracteres que actúa como clave.
- value1[256] – cadena de caracteres asociada.
- N_value2 – número de elementos del vector (entre 1 y 32).
- value2 – puntero a un vector de floats alocado dinámicamente con malloc.
- value3 – estructura Paquete con tres enteros (x, y, z).
- siguiente – puntero al siguiente nodo.

Esta estructura no impone ningún límite en el número de tuplas almacenadas. Se mantiene un puntero global head al primer nodo (inicialmente NULL). Todas las operaciones están protegidas con un pthread_mutex_t para garantizar la atomicidad requerida por el enunciado.

1.2 Versión distribuida (Parte B)

El sistema distribuido sigue el esquema cliente-servidor usando exclusivamente colas de mensajes POSIX (mqueue.h):

- Servidor (servidor-mq.c): crea una cola bien conocida (/servidor_mq) y entra en un bucle infinito. Por cada petición recibida, lanza un hilo POSIX desvinculado (PTHREAD_CREATE_DETACHED) que invoca la función correspondiente de libclaves.so y envía la respuesta a la cola del cliente.
- Proxy (proxy-mq.c): implementa la misma API que claves.h. Cada llamada abre la cola del servidor, crea una cola temporal de respuesta identificada por el PID del proceso, envía la petición y espera la respuesta. Devuelve -1 ante errores de validación y -2 ante errores de comunicación.
- Mensajes comunes (mq_common.h): define los tipos request_t y response_t, los códigos de operación y el nombre de la cola del servidor, compartidos por ambos lados.

El flujo de una petición es el siguiente:

```
app-cliente  →  proxy-mq  →  [cola /servidor_mq]  →  servidor-mq
                                        ↓ hilo
app-cliente  ←  proxy-mq  ←  [cola /cliente_PID]  ←  servidor-mq
```

2. Compilación y generación de ejecutables

Todos los artefactos se generan con el Makefile incluido. Requisitos previos: sistema Linux (o WSL) con gcc y soporte para colas de mensajes POSIX.

2.1 Compilar todo

```
make all
```

Esto genera:

Artefacto	Descripción
libclaves.so	Biblioteca dinámica – versión no distribuida
libproxyclaves.so	Biblioteca dinámica – lado cliente distribuido
app_cliente	Ejecutable cliente enlazado con libclaves.so
servidor_mq	Ejecutable del servidor de colas
app_cliente_mq	Ejecutable cliente enlazado con libproxyclaves.so

2.2 Compilar solo la parte A

```
make parte_a
```

2.3 Compilar solo la parte B

```
make parte_b
```

2.4 Limpiar

```
make clean
```

3. Plan de pruebas

El plan de pruebas se ejecuta con app-cliente.c, el cual es idéntico en la versión no distribuida y en la distribuida (solo varía la biblioteca enlazada). Las pruebas cubren todos los servicios de la API y comprueban tanto el camino exitoso como los casos de error.

#	Función	Descripción del test	Resultado esperado
1	destroy	Llamada inicial para limpiar el servicio	0
2	set_value	Insertar tupla válida (clave1)	0

3	set_value	Insertar tupla válida (clave2)	0
4	set_value	Insertar clave duplicada (clave1)	-1
5	set_value	N_value2 = 0 (fuera de rango)	-1
6	set_value	N_value2 = 33 (fuera de rango)	-1
7	exist	Clave que existe (clave1)	1
8	exist	Clave que no existe	0
9	get_value	Obtener valores de clave existente (clave1)	0 + valores correctos
10	get_value	Obtener valores de clave inexistente	-1
11	modify_value	Modificar clave existente (clave1)	0
12	get_value	Verificar que los valores se modificaron	0 + nuevos valores
13	modify_value	Modificar clave inexistente	-1
14	modify_value	N_value2 = 0 (fuera de rango)	-1
15	delete_key	Borrar clave existente (clave2)	0
16	exist	Verificar que clave2 ya no existe	0
17	delete_key	Borrar clave inexistente	-1
18	destroy	Destruir todas las tuplas	0
19	exist	Verificar que clave1 ya no existe tras destroy	0

```
pablo@OrdenadorPablo:/mnt/c/Users/pablo/Desktop/ejercicio_evaluable1$ make parte_a
gcc -Wall -fPIC -shared -o libclaves.so claves.c -lpthread
gcc -o app_cliente app-cliente.c -L. -lclaves -Wl,-rpath,.
pablo@OrdenadorPablo:/mnt/c/Users/pablo/Desktop/ejercicio_evaluable1$ ./app_cliente
=== destroy ===
destroy()          -> 0 (esperado 0)

=== set_value ===
set_value clave1   -> 0 (esperado 0)
set_value clave2   -> 0 (esperado 0)
set_value duplicada -> -1 (esperado -1)
set_value N=0      -> -1 (esperado -1)
set_value N=33     -> -1 (esperado -1)

=== exist ===
exist clave1       -> 1 (esperado 1)
exist no_existe    -> 0 (esperado 0)

=== get_value ===
get_value clave1   -> 0 (esperado 0)
  value1=valor1_de_clave1
  N_value2=3
  V_value2=1.00 2.00 3.00
  value3=(10,20,30)
get_value no_existe -> -1 (esperado -1)

=== modify_value ===
modify_value clave1 -> 0 (esperado 0)
  value1=valor1_modificado
  N_value2=4
  V_value2=9.90 8.80 7.70 6.60
  value3=(100,200,300)
modify_value no_existe -> -1 (esperado -1)
modify_value N=0   -> -1 (esperado -1)

=== delete_key ===
delete_key clave2  -> 0 (esperado 0)
exist clave2 tras borrar -> 0 (esperado 0)
delete_key no_existe -> -1 (esperado -1)

=== destroy final ===
destroy()          -> 0 (esperado 0)
exist clave1 tras destroy -> 0 (esperado 0)
pablo@OrdenadorPablo:/mnt/c/Users/pablo/Desktop/ejercicio_evaluable1$ make parte_b
gcc -Wall -fPIC -shared -o libproxyclaves.so proxy-mq.c -lrt
gcc -o servidor_mq servidor-mq.c -L. -lclaves -Wl,-rpath,. -lpthread -lrt
gcc -o app_cliente_mq app-cliente.c -L. -lproxyclaves -Wl,-rpath,.
pablo@OrdenadorPablo:/mnt/c/Users/pablo/Desktop/ejercicio_evaluable1$ ./servidor_mq
Servidor escuchando en /servidor_mq...
```

```
pablo@OrdenadorPablo:/mnt/c/users/pablo/desktop/ejercicio_evaluable1$ ./app_cliente_mq
=== destroy ===
destroy()          -> 0 (esperado 0)

=== set_value ===
set_value clave1   -> 0 (esperado 0)
set_value clave2   -> 0 (esperado 0)
set_value duplicada -> -1 (esperado -1)
set_value N=0      -> -1 (esperado -1)
set_value N=33     -> -1 (esperado -1)

=== exist ===
exist clave1       -> 1 (esperado 1)
exist no_existe    -> 0 (esperado 0)

=== get_value ===
get_value clave1   -> 0 (esperado 0)
  value1=valor1_de_clave1
  N_value2=3
  V_value2=1.00 2.00 3.00
  value3=(10,20,30)
get_value no_existe -> -1 (esperado -1)

=== modify_value ===
modify_value clave1 -> 0 (esperado 0)
  value1=valor1_modificado
  N_value2=4
  V_value2=9.90 8.80 7.70 6.60
  value3=(100,200,300)
modify_value no_existe -> -1 (esperado -1)
modify_value N=0   -> -1 (esperado -1)

=== delete_key ===
delete_key clave2  -> 0 (esperado 0)
exist clave2 tras borrar -> 0 (esperado 0)
delete_key no_existe -> -1 (esperado -1)

=== destroy final ===
destroy()          -> 0 (esperado 0)
exist clave1 tras destroy -> 0 (esperado 0)
```

3.1 Ejecución – Versión no distribuida (Parte A)
Desde el directorio del proyecto:
make parte_a
./app_cliente

La salida esperada muestra cada prueba con su resultado y el valor esperado entre paréntesis.

3.2 Ejecución – Versión distribuida (Parte B)
Terminal 1 – arrancar el servidor:
make parte_b
./servidor_mq

Terminal 2 – ejecutar el cliente:
./app_cliente_mq

El cliente es idéntico al de la Parte A. La única diferencia es que está enlazado con libproxyclaves.so, por lo que las llamadas a la API se traducen internamente en mensajes POSIX enviados al servidor.

Si las colas quedan en estado inconsistente entre ejecuciones, se pueden limpiar con:

ls /dev/mqueue/
rm /dev/mqueue/servidor_mq
rm /dev/mqueue/cliente_*